

WEBX Module 5

AngularJS Custom Directives

Custom directives let you create reusable HTML components.

Example: A custom `<user-card>` directive

```
// JS
angular.module('app', [])
.directive('userCard', function() {
  return {
    restrict: 'E',
    scope: { name: '@', role: '@' },
    template: '<div class="card"><h3>{{name}}</h3><p>{{role}}</p></div>',
    replace: true
  };
});

// HTML
<div ng-app="app">
  <user-card name="Alice" role="Admin"></user-card>
</div>
```

- `restrict: 'E'` → Element, also 'A'(attribute), 'C'(class), 'M'(comment)
 - `scope: {}` → isolate scope
 - `@` → one-way binding from parent
-

Explain controllers in Angular JS with example.

Controllers manage data and behavior for a view.

Example:

```
// JS
angular.module('app', [])
.controller('MainCtrl', function($scope) {
  $scope.message = 'Hello Angular';
  $scope.reverseMessage = function() {
    $scope.message = $scope.message.split('').reverse().join('');
  };
});
```

```

    };
  });

// HTML
<div ng-app="app" ng-controller="MainCtrl">
  <p>{{message}}</p>
  <button ng-click="reverseMessage()">Reverse</button>
</div>

```

- `$scope` connects controller to view
- Avoid heavy DOM logic in controllers (use directives instead)

Explain Routing using ng-Route, ng-Repeat, ng-style, ng-view with suitable example.

Routing creates single-page apps with multiple views.

Example:

```

// JS
angular.module('app', ['ngRoute'])
.config(function($routeProvider) {
  $routeProvider
    .when('/users', {
      templateUrl: 'users.html',
      controller: 'UsersCtrl'
    })
    .otherwise({ redirectTo: '/users' });
})
.controller('UsersCtrl', function($scope, $http) {
  $http.get('users.json').then(function(res) {
    $scope.users = res.data;
  });
});

// users.html
<div>
  <div ng-repeat="u in users" ng-style="{background: $index%2==0?'#eee':'white'}">
    {{u.name}}
  </div>
</div>

// Main HTML

```

```
<div ng-app="app">
  <a href="#/users">Users</a>
  <ng-view></ng-view>
</div>
```

Directive	Purpose
ngRoute	Routing module
ngView	Renders current route's template
ngRepeat	Loops over array
ngStyle	Dynamic CSS properties

Explain AngularJS \$http service in detail with its get() and post() methods.

`$http` makes AJAX requests. Returns a promise.

GET example:

```
$http.get('/api/users')
  .then(function(response) {
    $scope.users = response.data;
  })
  .catch(function(error) {
    console.error(error);
  });
```

POST example:

```
$http.post('/api/users', { name: 'John', age: 30 })
  .then(function(res) {
    $scope.result = res.data;
  })
  .catch(function(err) {
    $scope.error = err.statusText;
  });
```

Full config form:

```
$http({
  method: 'POST',
  url: '/api/users',
  data: { name: 'John' },
  headers: { 'Content-Type': 'application/json' }
})
```

- `get(url, config)` → simple GET
- `post(url, data, config)` → sends data
- Returns promise with `.then(success, error)`

Explain AngularJS filters with example.

Filters format data for display.

Built-in filters: `currency`, `date`, `filter`, `lowercase`, `uppercase`, `limitTo`, `orderBy`, `number`

Example:

```
// JS
angular.module('app', [])
.controller('ShopCtrl', function($scope) {
  $scope.products = [
    { name: 'Laptop', price: 999.99, date: new Date() },
    { name: 'Mouse', price: 19.50, date: new Date() }
  ];
});

// HTML
<div ng-app="app" ng-controller="ShopCtrl">
  <input ng-model="search">
  <ul>
    <li ng-repeat="p in products | filter:search | orderBy:'price'">
      {{p.name | uppercase}} - {{p.price | currency:'€'}} - {{p.date |
date:'dd/MM/yyyy'}}
    </li>
  </ul>
</div>
```

Custom filter example:

```
.filter('capitalize', function() {  
  return function(input) {  
    return input ? input.charAt(0).toUpperCase() + input.slice(1) : '';  
  };  
});
```

What is AngularJS dependency injection? State 6 examples of AngularJS built-in helper functions.

DI allows injecting dependencies (services, factories, etc.) into components.

```
angular.module('app', [])  
.controller('DemoCtrl', function($scope, $http, $timeout, $log, $window, $location)  
{  
  $http.get('/data');           // 1  
  $timeout(function() {}, 1000); // 2  
  $log.log('message');          // 3  
  $window.alert('Hi');          // 4  
  $location.path('/home');       // 5  
  $scope.$apply();              // 6  
});
```

6 Built-in helper functions:

Helper	Purpose
<code>angular.copy()</code>	Deep copy object
<code>angular.extend()</code>	Merge objects
<code>angular.equals()</code>	Deep equality check
<code>angular.forEach()</code>	Iterate over collections
<code>angular.toJson()</code>	Object → JSON string
<code>angular.fromJson()</code>	JSON string → object

DI makes code testable and modular.

Explain the following built-in AngularJS Directives with example: a) ng-app b) ng-init.

Directive	Purpose
ng-app	Auto-bootstraps Angular app
ng-init	Initializes scope variables (avoid in production)
ng-model	Two-way binds input to scope

Example:

```
<div ng-app="myApp" ng-init="count=5">
  <p>Initial count: {{count}}</p>
  <input type="text" ng-model="name" placeholder="Enter name">
  <p>Hello, {{name}}!</p>
  <button ng-click="count = count + 1">Add</button>
  <p>Count: {{count}}</p>
</div>
```

```
angular.module('myApp', []);
```

- ng-model works on input, select, textarea
- ng-init okay for demos, not real apps
- ng-app can specify module name

Explain Expressions in AngularJS

Expressions are JavaScript-like code inside `{{ }}`.

```
<p>{{ 5 + 5 }}</p>           <!-- 10 -->
<p>{{ firstName + ' ' + lastName }}</p>
<p>{{ user.age > 18 ? 'Adult' : 'Minor' }}</p>
<p>{{ items.length }}</p>
<p>{{ getFullName() }}</p>
```

Differences from JS:

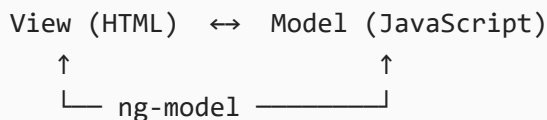
- No loops or conditionals (if , for) – use ng-if , ng-repeat

- No function declarations
 - No variable creation (`var x = ...` not allowed)
 - Handles `undefined / null` gracefully (shows empty)
 - Filters can be chained: `{{ name | uppercase | limitTo:10 }}`
-

Write a Short Note on AngularJS Data Binding

AngularJS supports two-way data binding – changes in UI update model, changes in model update UI instantly.

Two-way binding diagram:



Example:

```
<input ng-model="username">
<p>Hello {{username}}</p>
```

Type in input → `{{username}}` updates → change `$scope.username` in code → input value updates.

One-way binding options:

- Interpolation: `{{::value}}` (one-time)
- `ng-bind`: `<span ng-bind="data"></span>`

No manual DOM updates or event listeners needed.

Explain AngularJS Scope

`$scope` is the glue between controller and view – a plain JS object.

Types:

- `$rootScope` – global scope (avoid overuse)
- Child scope – created by controllers, directives

- Isolate scope – custom directives with `scope: {}`

Example:

```
.controller('ParentCtrl', function($scope) {
  $scope.parentName = 'John';
})
.controller('ChildCtrl', function($scope) {
  $scope.childName = 'Tim';
});
```

Inheritance: Child scopes prototypically inherit from parent.

```
<div ng-controller="ParentCtrl">
  {{parentName}}
  <div ng-controller="ChildCtrl">
    {{parentName}} - {{childName}} <!-- John - Tim -->
  </div>
</div>
```

- `$scope.$watch()` – listens for changes
- `$scope.$apply()` – triggers digest cycle
- `$scope.$emit()` / `$broadcast()` – event propagation

List the features of AngularJS. Also state its advantages and disadvantages.

Features:

- Two-way data binding
- MVC/MVW architecture
- Dependency Injection
- Directives (reusable components)
- Routing (SPA)
- Templates with expressions
- Form validation
- Testing ready (mocks)

Advantages:

Area	Benefit
Productivity	Less boilerplate code
Maintainability	Separation of concerns
Testing	DI makes unit testing easy
Community	Large ecosystem, many resources
DOM manipulation	Minimal – directives handle it

Disadvantages:

Issue	Explanation
Performance	Dirty checking (digest cycle) slows with many watchers
Learning curve	Concepts like DI, digest, scopes
Migration	AngularJS (1.x) → Angular (2+) is complete rewrite
Debugging	Templates errors can be cryptic
Security	Must sanitize user input to prevent XSS
SEO	Poor for crawlers (pre-rendering needed)